RICE UNIVERSITY

Multi-Server Communication

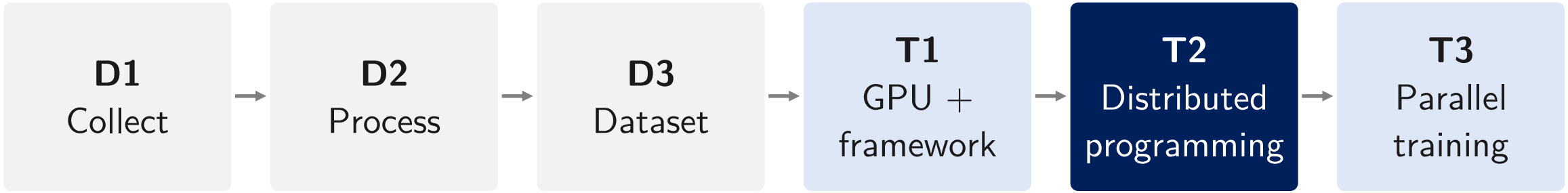
Topology, launching, hangs, and an α–β model of the network between servers

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 11, Lecture 2

Where we are in the pipeline



In our pipeline

Today: collectives **across servers**, and the guided **two-server** benchmark with an α–β **fit** (Lab 4, due after this lecture).

Last time: NCCL, ring and tree all-reduce, busbw, streams and ordering, on one server.

Learning goals for today

- Describe the **topology** inside a server (NVLink, NVSwitch, PCIe) and between servers (InfiniBand, RoCE).
- Explain why the **slowest link** sets the pace, and how **hierarchical** collectives help.
- **Launch** a job on two servers with torchrun or mpiexec, and map ranks to GPUs.
- Diagnose **timeouts and hangs**: mismatched calls, crashed ranks, wrong interfaces.
- **Fit an α–β model** to intra- and inter-server measurements and use it to predict.

A puzzle to start

Each GPU holds 1 GB of gradients; all-reduce them.

A: 8 GPUs in one server (NVLink). B: 16 GPUs in two servers (plus the network).

Is B faster, about the same, or slower than A?

Key idea

Slower. A $\approx$ 3.9 ms. B with one ring through the network $\approx$ **37.5 ms** ($\approx 10\times$). A topology-aware algorithm gets B to $\approx$ 6.4 ms.

Source: Model estimates. NVLink: H100 datasheet; network: 400 Gb/s NDR InfiniBand NIC (typical).

Recap: ring cost and bus bandwidth

$$T = 2(p-1)\alpha + 2(p-1)/p \cdot n \;/\; \beta$$

$$\text{busbw} = (n \;/\; t) \cdot 2(p-1)/p$$

Watch out

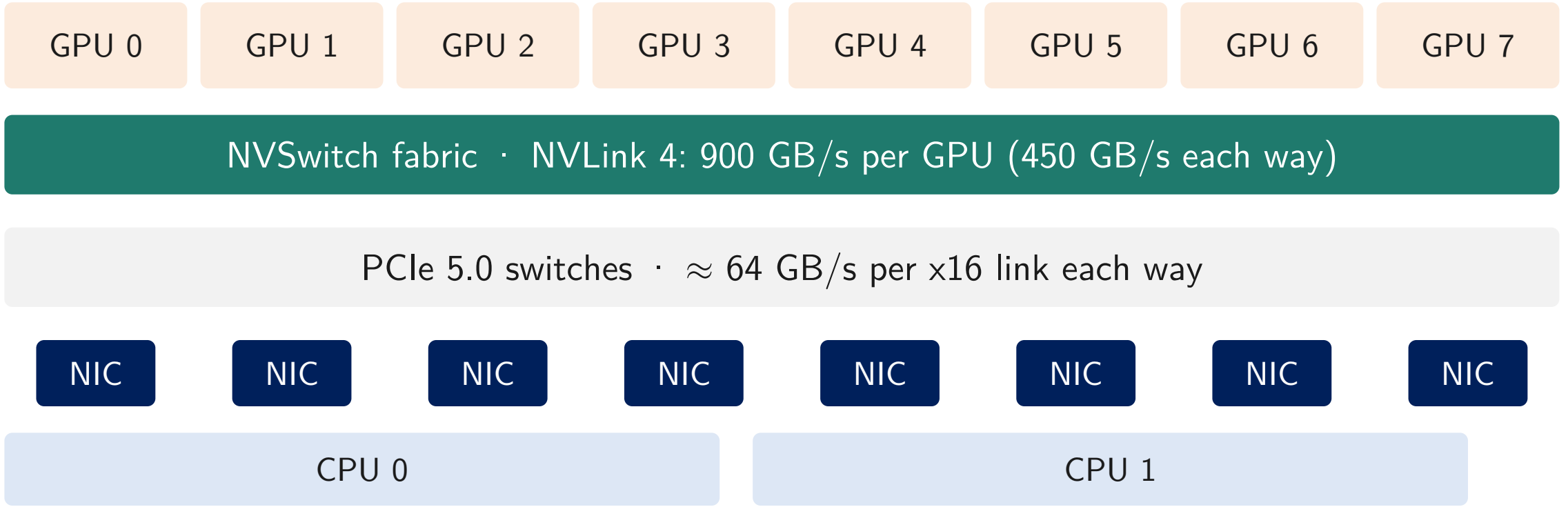
Last lecture every link had the same β. Across servers they **do not** — which β do we use?

Part 1

Topology

What connects the GPUs, inside a server and between servers?

Inside one server



Key idea

GPU $\leftrightarrow$ GPU inside the server: **NVLink**. Everything else (CPU, NIC) goes through **PCIe**.

Source: NVIDIA DGX H100 datasheet and user guide: 8 × H100, 4 NVSwitch chips, 8 × ConnectX-7 at 400 Gb/s. NVLink: H100 datasheet.

Seeing the topology: nvidia-smi topo -m

```
$ nvidia-smi topo -m          # abridged
        GPU0  GPU1  GPU2  GPU3  NIC0
GPU0     X    NV18  NV18  NV18  PIX
GPU1    NV18   X    NV18  NV18  NODE
GPU2    NV18  NV18   X    NV18  SYS
GPU3    NV18  NV18  NV18   X    SYS
```

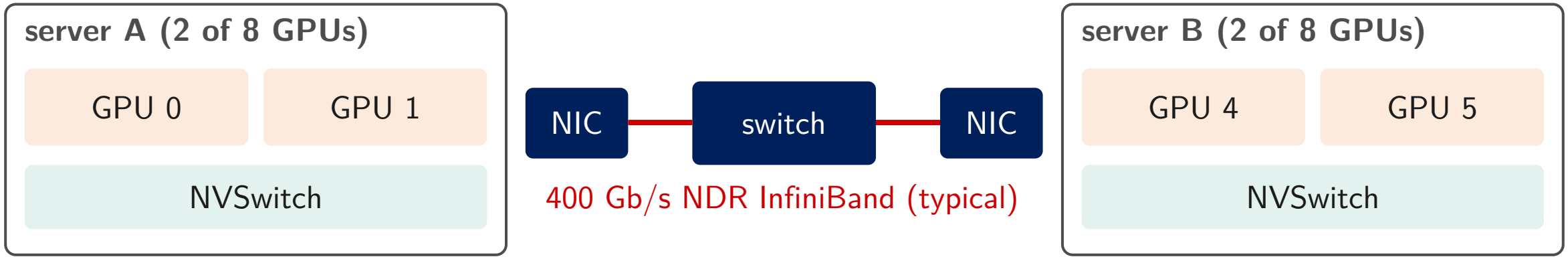
Code	Meaning
NV18	18 NVLinks
PIX	one PCIe switch
NODE	same CPU socket
SYS	across CPU sockets

Read it like this

GPU0 should use **NIC0** (PIX); a GPU whose nearest NIC is **SYS** pays a slow detour.

Source: nvidia-smi topo -m legend; illustrative output for an 8-GPU H100 server.

Between servers: NICs and switches



- **InfiniBand** or **RoCE**: RDMA — the NIC writes remote memory without the CPU.
- **GPUDirect RDMA**: the NIC reads GPU memory directly over PCIe.
- No RDMA? NCCL falls back to **TCP sockets**: slower, and costs CPU time.

Source: NVIDIA DGX H100 datasheet and user guide: 8 × H100, 4 NVSwitch chips, 8 × ConnectX-7 at 400 Gb/s. NCCL User Guide.

Bits, bytes, and the gap

Link (one direction)	Nominal rate	In GB/s
NVLink 4, per H100 GPU	—	450
PCIe 5.0 x16	—	≈ 64
NDR InfiniBand, one NIC	400 Gb/s	50
HDR InfiniBand, one NIC	200 Gb/s	25
100 Gb Ethernet (TCP)	100 Gb/s	12.5

$$400\ \mathrm{Gb/s} \div 8 = 50\ \mathrm{GB/s}$$

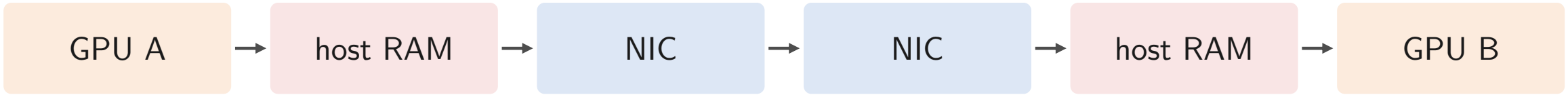
Watch out

NVLink ÷ NIC = 450 ÷ 50 = **9×**. Leaving the server is the expensive step.

Source: Typical, rounded line rates. InfiniBand Trade Association roadmap (NDR = 400 Gb/s per port); PCI-SIG PCIe 5.0; H100 datasheet.

The path of one message between servers

Without GPUDirect RDMA



two extra copies through host memory; the CPU is involved

With GPUDirect RDMA



NIC reads and writes GPU memory directly over PCIe

Key idea

NCCL uses GPUDirect RDMA automatically when the drivers support it; the debug log says so.

Source: NVIDIA GPUDirect RDMA documentation; NVIDIA NCCL User Guide: Environment Variables; Troubleshooting.

Check yourself

Two minutes with a neighbour.

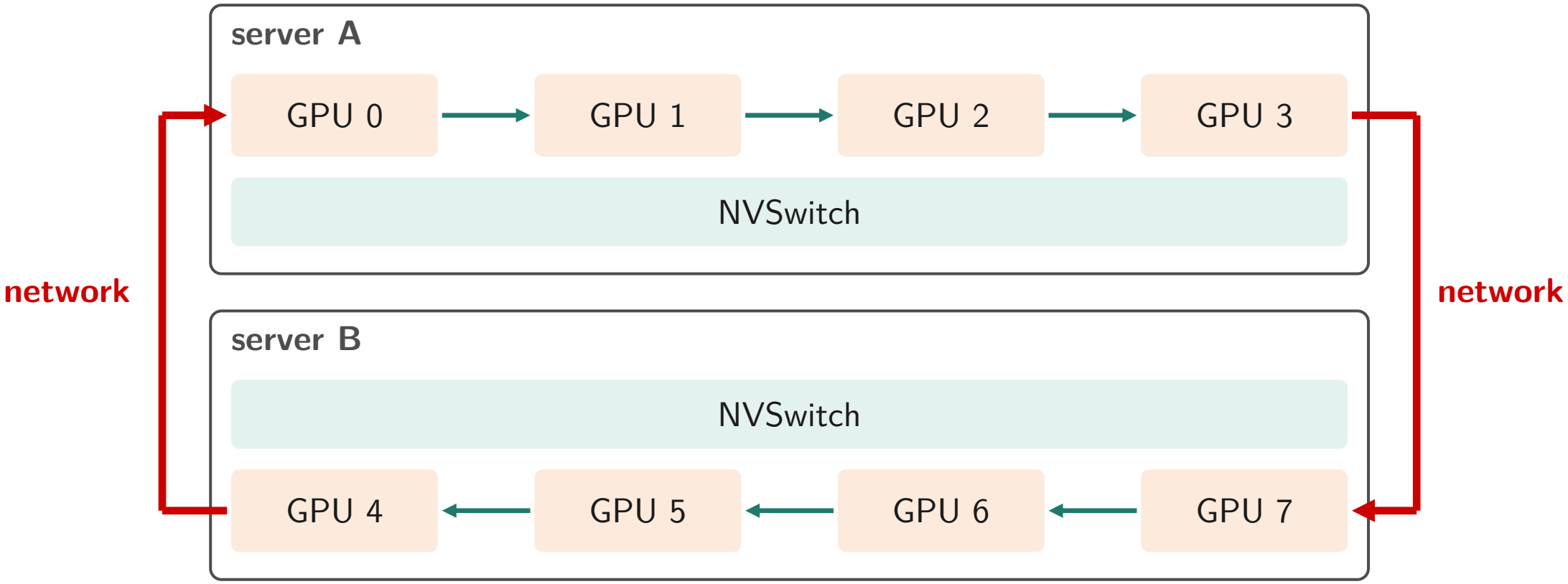
- A cluster has 200 Gb/s HDR NICs. How many GB/s is that? How many times slower than NVLink?
- In `nvidia-smi topo -m`, what do **NV18** and **SYS** mean?
- What does GPUDirect RDMA save?

Part 2

Hierarchical collectives

If one link is 9× slower, how should the algorithm change?

A flat ring across two servers



Watch out

Each ring step ends when its **slowest hop** ends: the whole ring runs at network speed.

The slowest link sets the pace

$$\text{one step} = \max \text{ over links } (\ \alpha + (n/\text{p})\ /\ \beta(\text{link})\ )$$

$$T \approx 2(p-1)\ \alpha + 2(p-1)/p \cdot n\ /\ \beta(\text{slowest})$$

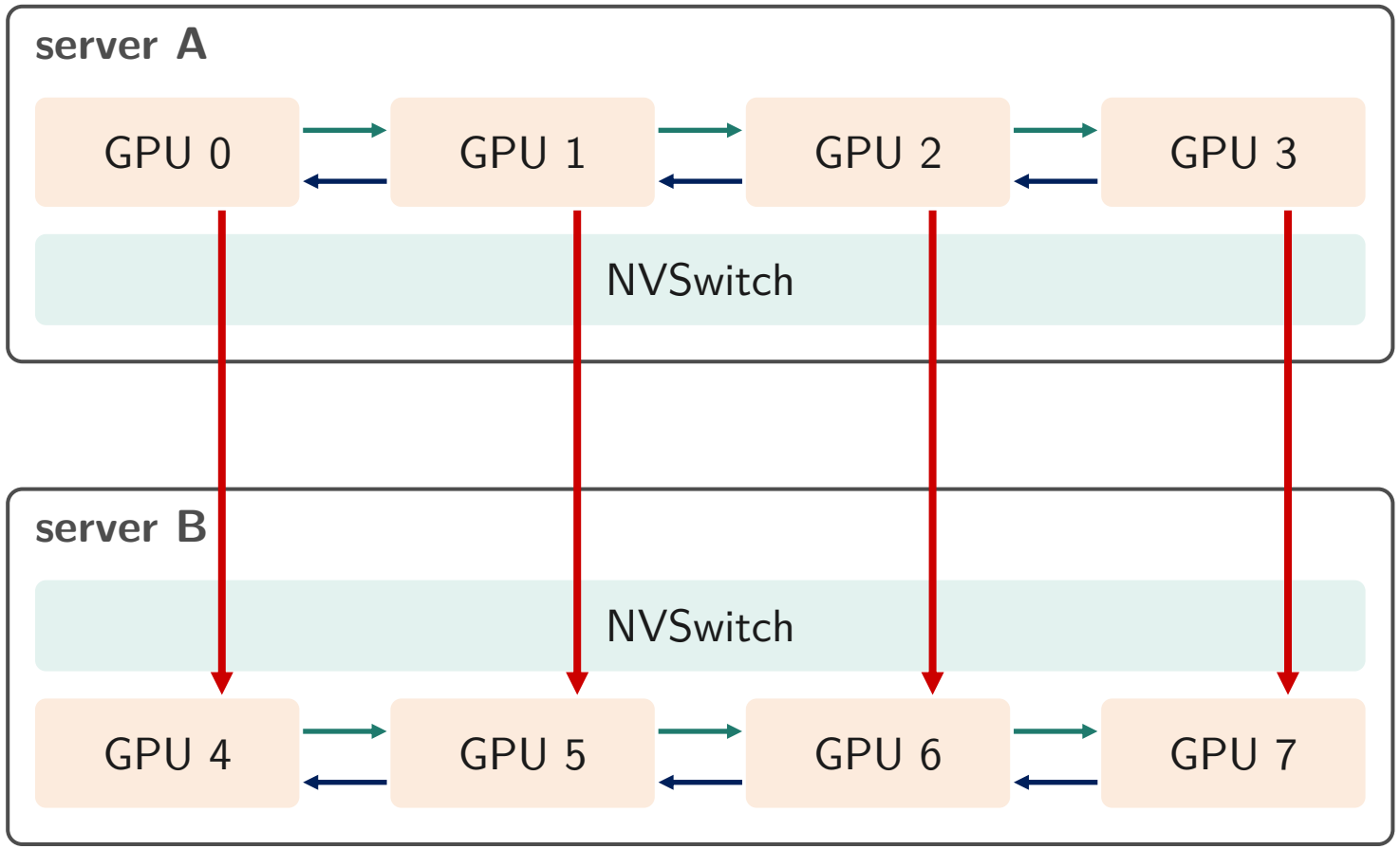
$$p = 16,\ n = 1\ \text{GB},\ \beta = 50\ \text{GB/s}:\ \ T \approx 37.5\ \text{ms}$$

Watch out

One server, 8 GPUs: $\approx$ 3.9 ms. Adding a server made the same job $\approx$ **10× slower**.

Source: Model estimate; NIC: 400 Gb/s NDR (typical); NVLink: H100 datasheet.

Hierarchical all-reduce: three phases



1 · reduce-scatter inside each server (NVLink)

2 · all-reduce slice k between GPU k's, over g NICs in parallel

3 · all-gather inside each server (NVLink)

Key idea

The network carries only **n/g bytes per GPU**, on g NICs at once.

Cost of the hierarchical all-reduce

$$T \approx 2 \cdot (g-1)/g \cdot n \;/\; \beta(\text{in}) \;+\; 2(N-1)/N \cdot (n \;/\; g) \;/\; \beta(\text{out})$$

$$= 2 \times 1.94\ \text{ms} \;+\; 2.50\ \text{ms} \;\approx\; 6.4\ \text{ms}$$

g = 8 GPUs per server, N = 2 servers, n = 1 GB, β(in) = 450 GB/s, β(out) = 50 GB/s

Why it works

The network term is divided by **g**: each NIC carries only its GPU's 1/g slice.

Source: Model estimate; latency terms omitted. NVLink: H100 datasheet; NIC: 400 Gb/s NDR (typical).

Same job, four layouts

Layout (1 GB all-reduce)	Bottleneck	Model time
1 server $\times$ 8 GPUs	NVLink	$\approx$ 3.9 ms
2 servers, flat ring	one NIC hop	$\approx$ 37.5 ms
2 servers, hierarchical, 8 NICs each	8 NICs in parallel	$\approx$ 6.4 ms
2 servers, hierarchical, 1 NIC each	one shared NIC	$\approx$ 23.9 ms

Key idea

Topology-aware algorithms close most of the gap — **if** each GPU has its own path to the network.

Source: Model estimates. NVLink: H100 datasheet; NIC: 400 Gb/s NDR (typical); DGX H100 datasheet.

Check yourself

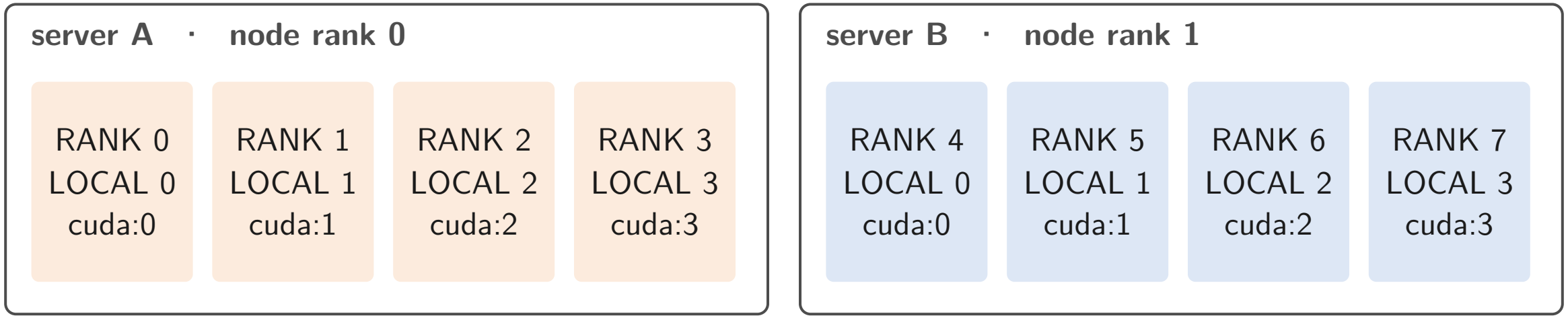
- Redo the hierarchical estimate for 2 servers with **200 Gb/s** NICs (one per GPU).
- Why does a flat ring spanning two servers waste NVLink?
- Which traffic would you keep **inside** a server: the frequent or the rare?

Part 3

Launching multi-node jobs

How do processes on two machines find each other?

What every process must know



$$\text{RANK} = \text{node rank} \times 4 + \text{LOCAL_RANK}$$

Key idea

Shared by all: `WORLD_SIZE=8`, `MASTER_ADDR` = address of rank 0's host, `MASTER_PORT` = a free port.

Source: PyTorch documentation, torch.distributed and torchrun (v2.9).

torchrun on two servers

```
# run on BOTH servers (same command)
torchrun --nnodes=2 --nproc-per-node=4 \
  --rdzv-id=lab4 --rdzv-backend=c10d \
  --rdzv-endpoint=node01:29500 \
  bench_allreduce.py
```

- `--nnodes`: servers
- `--nproc-per-node`: GPUs
- `--rdzv-endpoint`: meeting point
- `--rdzv-id`: one job id

WHAT HAPPENS

torchrun waits at rendezvous until **both** servers join; then it sets `RANK`, `LOCAL_RANK`, `WORLD_SIZE`, `MASTER_ADDR`, `MASTER_PORT`.

Source: PyTorch documentation, torch.distributed and torchrun (v2.9).

Inside the script

start of the benchmark script

```
import os, socket, datetime, torch
import torch.distributed as dist

local = int(os.environ["LOCAL_RANK"])
torch.cuda.set_device(local)
dist.init_process_group(
    backend="nccl",
    timeout=datetime.timedelta(minutes=5))
rank = dist.get_rank()          # 0 .. 7
print(rank, local, socket.gethostname(),
      flush=True)
```

- Same file, any server count
- **Short timeout** to debug
- Print rank, host, GPU

LAB 4

Lab 4, step 1: a table of `RANK`, `LOCAL_RANK`, host, and GPU for every process.

Source: PyTorch documentation, torch.distributed and torchrun (v2.9).

mpiexec with a host file

```
# file: hosts
node01 slots=4
node02 slots=4

mpiexec -n 8 --hostfile hosts \
  -x MASTER_ADDR=node01 -x MASTER_PORT=29500 \
  python bench_allreduce.py
```

```
e = os.environ
rank = int(e["OMPI_COMM_WORLD_RANK"])
local = int(e["OMPI_COMM_WORLD_LOCAL_RANK"])
p = int(e["OMPI_COMM_WORLD_SIZE"])
torch.cuda.set_device(local)
dist.init_process_group("nccl", rank=rank,
                        world_size=p)
```

- Open MPI: `--hostfile`, `-x`
- MPICH: `-f`, `-genv`
- MPI only **launches**;
- **NCCL** still moves GPU data.

Source: Open MPI mpirun(1) manual; PyTorch documentation, torch.distributed and torchrun (v2.9).

The same environment on every host

Must match on every host	If not...
Python env, PyTorch, CUDA, NCCL versions	import errors, crashes, hangs at init
Code version	different collective order → hang or wrong sums
Data path and number of batches per rank	extra collective on one rank → hang
Host names, open ports, network interface	rendezvous or NCCL init times out

Tip

Use a **shared file system** or one **container image** for all hosts.

Check yourself

- 4 processes per server. Which RANK and GPU does local rank 3 on node 1 get?
- You start torchrun on node01 but forget node02. What happens?
- Why should `--nnodes` and `--nproc-per-node` be the same on both servers?

Part 4

Timeouts and hangs

Why does my job sit there doing nothing?

Three kinds of timeout

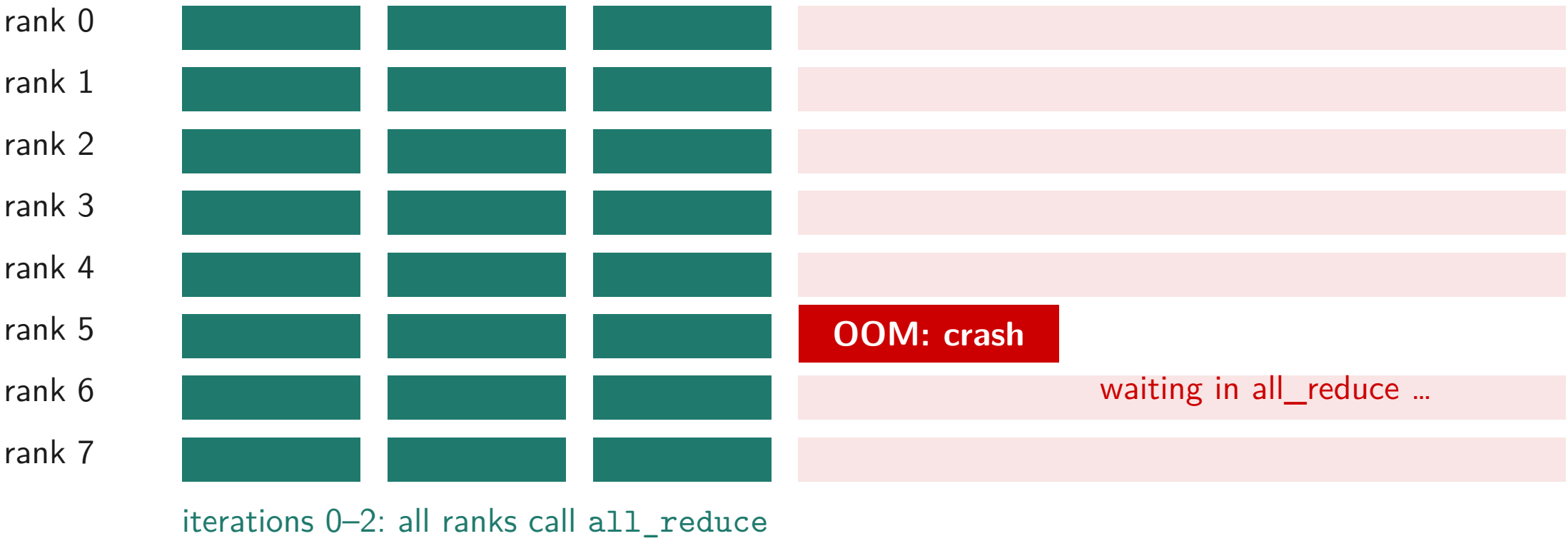
Where it waits	Symptom	Controlled by
Rendezvous / init	nothing after start-up	launcher, `init_process_group`
A collective	it never completes	`timeout=` (NCCL default 10 min)
NCCL watchdog	"Watchdog caught collective operation timeout"	same timeout; aborts the process

Watch out

NCCL calls return immediately, so a **watchdog thread** must detect a stuck collective — after the timeout.

Source: PyTorch documentation, torch.distributed and torchrun (v2.9).

Hang 1: a crashed rank



Watch out

Read the log of the **first rank that failed**; the others only report the timeout.

Hang 2: mismatched collectives

ranks 0–6

```
for step in range(steps):
    loss = train_step()
    dist.all_reduce(grads)
```

rank 7 (stale code on node02)

```
for step in range(steps):
    loss = train_step()
    dist.all_reduce(loss)  # extra
    dist.all_reduce(grads)
```

- Rank 7's **loss** is paired with everyone else's **grads**: wrong sizes $\rightarrow$ hang.
- Other causes: unequal **data shards**, conditions on **local** state (files, host names).

Key idea

Find **which call** each rank is in: print a counter, or `TORCH_DISTRIBUTED_DEBUG=DETAIL`.

Source: PyTorch documentation, torch.distributed and torchrun (v2.9).

Hang 3: the wrong network interface

server A: interfaces

lo
loopback: reaches only itself

docker0
virtual bridge: wrong network

eth0
Ethernet: reaches server B

ib0 / mlx5_0
InfiniBand: the fast path

```
export NCCL_SOCKET_IFNAME=eth0
export GLOO_SOCKET_IFNAME=eth0
export NCCL_IB_HCA=mlx5      # IB adapters
```

- Set them **the same** on every host.
- `^docker0,lo` means "anything except these".
- Firewalls: open TCP between hosts.

Source: NVIDIA NCCL User Guide: Environment Variables; Troubleshooting.

Reading NCCL_DEBUG=INFO

abridged; wording varies by NCCL version

```
NCCL INFO Bootstrap : Using eth0:10.0.0.1
NCCL INFO NET/IB : Using [0]mlx5_0:1/IB
NCCL INFO Channel 00/08 : 0 1 2 3 4 5 6 7
NCCL INFO ... via NET/IB/0/GDRDMA
NCCL INFO comm ... nranks 8 ... Init COMPLETE
```

You see	It means
`Using docker0` or `lo`	wrong interface: set `NCCL_SOCKET_IFNAME`
`NET/Socket` instead of `NET/IB`	no InfiniBand: TCP speed only
no `Init COMPLETE` on some rank	that rank never joined: start there

Source: NVIDIA NCCL User Guide: Environment Variables; Troubleshooting.

A debugging checklist

- **Start small**: 2 GPUs on 1 server $\rightarrow$ 1 GPU on each of 2 servers $\rightarrow$ all.
- **Print** RANK, LOCAL_RANK, host, GPU from every process.
- **NCCL_DEBUG=INFO**: interface? IB or sockets? Init COMPLETE on every rank?
- **Same software and code** on every host; same launcher flags.
- **Do all ranks reach the same call?** Count collectives per rank.
- On a timeout, read the **first rank that failed**.

Check yourself

- 7 ranks print "step 41", rank 2 printed nothing since step 40. Where do you look?
- Works on one server, hangs at init on two. Name three suspects.
- Inter-node bandwidth is only $\approx$ 3 GB/s and the log says `NET/Socket`. Why?

IN-CLASS BUILD

Two-server benchmark and α–β fit

Design it in pairs now; run it in Lab 4.

Exercise: design the two-server benchmark

Configuration	Processes	Link measured
intra	2 GPUs, one server	NVLink
inter	1 GPU on each of two servers	one NIC (InfiniBand or RoCE)

- **Sizes**: 16 B $\rightarrow$ 1 GB, powers of 4 (both flat and sloped regions).
- **Timing**: warm up, $\geq$ 20 iterations, synchronize, slowest rank; sweep 3 times.
- **Record**: size, median and spread, algbw, busbw, and the NCCL transport line.

Key idea

p = 2 in both runs: the **only** difference is the link, and busbw = algbw.

Fitting α and β

$$T(n) = \alpha + n \,/\, \beta$$

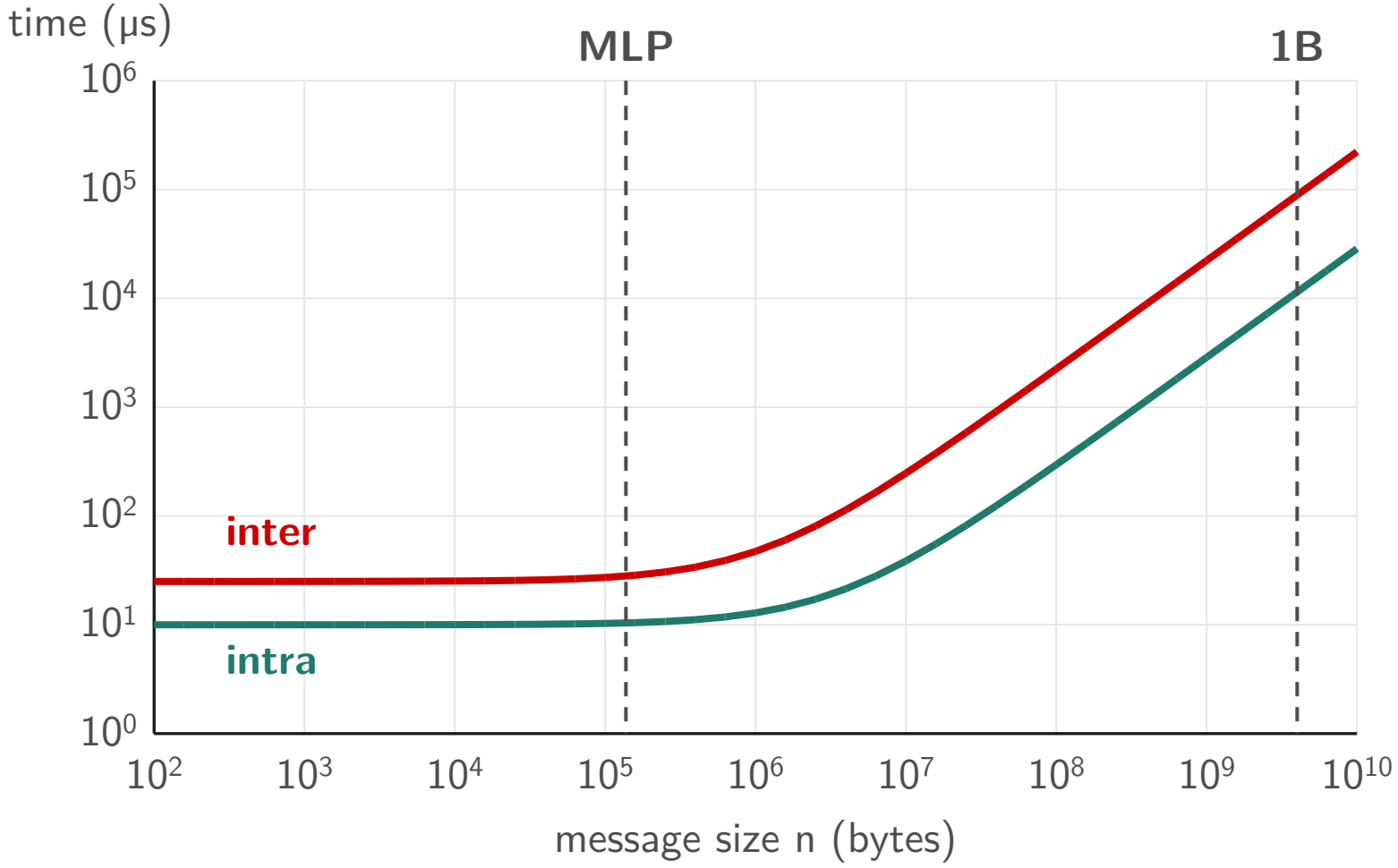
$$\text{minimize} \;\; \Sigma \left(\; (t - \alpha - n \,/\, \beta) \,/\, t \; \right)^2 \;\; \text{over all sizes}$$

```
n = np.array(sizes, float)          # bytes
t = np.array(times)                 # seconds
A = np.stack([np.ones_like(n), n], axis=1)
w = 1 / t                           # relative
(a, b), *_ = np.linalg.lstsq(A * w[:, None],
                             t * w, rcond=None)
alpha, beta = a, 1 / b
print(alpha * 1e6, "us", beta / 1e9, "GB/s")
```

Watch out

Unweighted least squares lets the **largest** messages decide, and α comes out wrong. Weight by 1/t.

What the two fits look like



Illustrative fits:

intra: $\alpha = 10$ µs, $\beta = 350$ GB/s

inter: $\alpha = 25$ µs, $\beta = 45$ GB/s

n* $= \alpha\beta$: 3.5 and 1.1 MB

Source: Model curves with assumed parameters, not measurements; replace with your Lab 4 fits.

Predict: our MLP and a larger model

Gradient all-reduce, p = 2	intra (NVLink)	inter (network)	inter ÷ intra
MLP: 137,352 bytes	10.4 µs	28.1 µs	2.7×
1B parameters: 4 GB	11.4 ms	88.9 ms	7.8×

- MLP: $n \ll n^* \rightarrow$ time $\approx \alpha$. The ratio is α(inter) / α(intra).
- 1B model: $n \gg n^* \rightarrow$ time $\approx$ **n** / β. The ratio is β(intra) / β(inter).

Key idea

Small models pay for **latency** (number of messages); large models pay for **bandwidth** (bytes).

Source: Predictions from the illustrative fits on the previous slide; MLP size from common.MODEL.

Interpreting the gap

Regime	Symptom	Lever (week)
Latency-bound	many small all-reduces	bucket gradients (12)
Bandwidth-bound	time $\propto$ bytes across servers	hierarchy, layout, precision (12–13)
Either	GPU idle during all-reduce	overlap with backward (12)

Key idea

Know which regime you are in **before** optimizing: n compared with $n^* = \alpha \cdot \beta$ tells you.

Lab 4: what to hand in

Part	Deliverable
Rank/device mapping	table: RANK, LOCAL_RANK, host, GPU for every process
CPU messages	mpi4py send/recv timing (Week 10)
NCCL collectives	correctness tests + busbw sweep on one server (Week 11 L1)
Guided two-server run	intra and inter sweeps, NCCL_DEBUG transport lines
α–β fit	α (µs), β (GB/s) for each; plot; MLP and 1B predictions

In our pipeline

Report the **gap** between intra and inter, and explain it with α, β, and the topology.

Key takeaways

- **NVLink** inside a server (450 GB/s); a **NIC** between servers (50 GB/s).
- A ring runs at its **slowest link**; **hierarchical** collectives send n/g per NIC.
- Every process needs `RANK`, `LOCAL_RANK`, `WORLD_SIZE`, `MASTER_ADDR`, `MASTER_PORT`.
- Hangs: **mismatched calls**, **crashed ranks**, **wrong interfaces**.
- Fit $\mathbf{T = \alpha + n/\beta}$ per link; $n^* = \alpha\beta$ separates latency- from bandwidth-bound.

Next time, and readings

When	What
After this lecture	Lab 4 due
Week 12	T2 due; T3 out
Week 12, Lecture 1	Data parallelism and sharding: DDP, ZeRO, FSDP
Week 12, Lecture 2	Tensor and model parallelism

READ BEFORE WEEK 12

PyTorch *Distributed Overview* and the DDP tutorial.

NCCL User Guide: *Environment Variables* and *Troubleshooting*.